

You’ve Got a Golden Ticket: Improving Generative Robot Policies With A Single Noise Vector

Omkar Patil^{1,2,*}, Ondrej Biza¹, Thomas Weng¹, Karl Schmeckpeper¹, Wil Thomason¹, Xiaohan Zhang¹, Kausik Sivakumar¹, Robin Walters^{1,3}, Nakul Gopalan², Sebastian Castro¹, Stephen Hart¹, Eric Rosen¹

Abstract—Generative robot policies trained on demonstrations using behavior cloning often learn actions that are sub-optimal or misaligned with respect to the downstream task. Policy improvement approaches aim to bridge this gap and improve the cumulative reward with minimal interventions on the pre-trained policy. However, an impediment to their practical deployment is the requirement of cumbersome hyperparameter tuning specific to model or task families. In this work, we present an episodic, derivative-free latent policy improvement approach that works with little to no changes across model scales from MLPs to VLAs. We demonstrate that the performance of a pretrained, frozen diffusion or flow matching policy can be improved with respect to a downstream reward by swapping the sampling of initial noise from the prior distribution (typically isotropic Gaussian) with a well-chosen, constant initial noise input—a *golden ticket*. We show the prevalence of golden tickets by improving the policy performance of 46 out of 51 tasks across manipulation benchmarks, with absolute improvements in success rate by up to 79% for simulated tasks, and 28% within 60 search episodes for real-world tasks. Further, we find that the versatility of our approach opens up new avenues such as simultaneous policy improvement for multiple downstream rewards. Project webpage: <https://lottery-tickets.rai-inst.com/>.

I. INTRODUCTION

Conditional diffusion [1] and flow matching models [2] are popular approaches for learning robot control policies, because of their ability to represent high-dimensional, multi-modal action distributions. However, these models often learn sub-optimal actions from noisy demonstrations or are not aligned with the downstream task-reward specification. Hence, generative policy improvement approaches [3]–[7] aim to improve the pretrained policy’s performance with respect to the downstream reward. However, currently improving their out-of-the-box performance introduces several computational and model design challenges. Existing methods for improving generative policies focus largely on derivative-based approaches: finetuning the pre-trained model weights [3], training an additional network [4], [8], or assuming specific network architectures [5] or training regimes [9]. These prior methods are computationally expensive, require complex model design choices and hyperparameter tuning (see Fig. 1), or require assumptions that limit the scenarios in which they can be applied.

In the fields of policy search [10] and optimization [11], [12], *derivative-free* methods provide an alternative approach to derivative-based methods such as actor-critic and policy gradients. For example, techniques such as CEM [12]

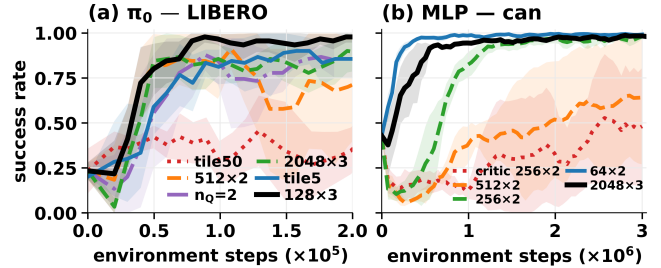


Fig. 1: Model design challenges for derivative-based methods: Two plots showing only DSRL curves with different hyperparameters. Actor-critic MLP used for Can task 128x3 is not optimal on LIBERO#57. (a) π_0 ’s larger action space needs 10 critics n_Q and noise space tiling. (b) Actor-critic MLP 64x2 beats the reported result (black) but 512x2 fails. *Can policy improvement be made simpler and robust?*

and CMA-ES [13] have long been used for policy search in robotics [10]. Derivative-free methods have the benefit of requiring only objective evaluations, enabling optimization over black-box, discontinuous, or otherwise non-differentiable systems [11]. Moreover, derivative-free methods such as ES [14] and ARS [15] have also been proposed as simpler and more robust alternatives to hyperparameter-sensitive approaches for learning control policies. We therefore ask whether episodic and derivative-free (black-box) approaches can be used for optimizing the performance of pretrained generative robot control policies. While prior derivative-free policy search methods [10] directly sample the space of model parameters, we instead focus on a feature of diffusion and flow matching: selecting the initial noise vector used for the trajectory denoising process.

In this work, we demonstrate that the performance of a pretrained, frozen diffusion or flow matching policy can be improved by simply replacing the sampling of initial noise from the prior distribution with a well-chosen, constant initial noise input, which we call a *golden ticket* (Fig. 2). Given a pretrained policy, a reward function, and an environment for a new downstream task, we pose finding golden tickets as a search problem, where candidate initial noise vectors, which we call *lottery tickets*, are optimized to find the ticket that maximizes the expected cumulative discounted rewards on the downstream task.

At first glance, one might expect using a fixed noise vector for denoising to perform worse than the standard approach of sampling independent noise vectors at each step. Our counterintuitive observation is that a *single, well-*

¹Robotics and AI Institute (RAI)

²Arizona State University (ASU)

³Northeastern University

^{*}Work done while interning at RAI.

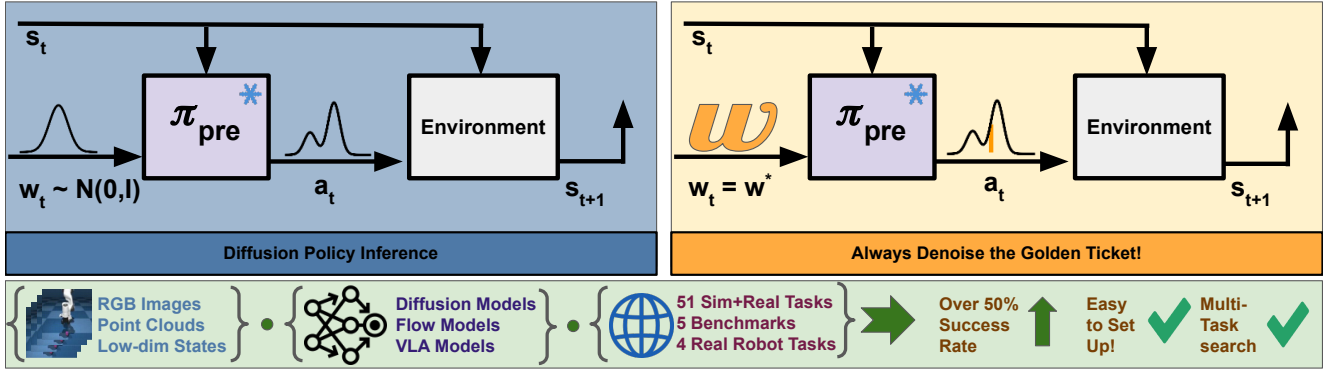


Fig. 2: Overview of standard diffusion policy inference (left) versus our proposed inference approach of using golden tickets (right). Given a frozen, pretrained diffusion or flow matching policy π_{pre} , rather than sampling from a Gaussian every time an action needs to be computed, we use a constant, well-chosen initial noise vector w , called a golden ticket. We find golden tickets improve policy performance across a range of observation inputs, model architectures, and embodiments.

chosen noise vector encodes critical information about how the policy should do the task. This phenomenon has been investigated in prior work on image generation [16]–[19], but its role in temporally conditioned generative robot policies is underexplored. DSRL [4] is the closest work in this direction, proposing a derivative-based method that learns to select diffusion noise inputs via reinforcement learning. However, while DSRL can achieve strong results, it is sensitive to hyperparameter tuning, does not scale well to large action spaces, and applies gradient-based learning to a highly non-smooth noise-selection landscape. Although individual noise vectors are smoothly transformed through the denoising process, nearby points in the initial noise space can map to substantially different actions. Noise selection is therefore a natural candidate for gradient-free optimization.

We propose Golden Ticket (GT) Search as a lightweight but surprisingly effective policy improvement approach that requires little-to-no hyperparameter tuning. GT Search is intended less as a universal policy improvement algorithm than as a simple method for probing how much performance can be recovered through direct search over latent noise. We show that searching for constant noise vectors in the policy’s latent space can yield comparable performance to gradient-based methods that learn state-conditioned noise predictors, especially for higher dimensional action spaces. GT Search keeps the pre-trained policy weights frozen, does not train an additional network, and can be applied to any diffusion or flow-matching framework without additional assumptions.

Our experimentation is designed to first empirically prove the existence and prevalence of golden tickets for a wide range of benchmarks and policies, following which we discuss the sample efficiency of searching golden tickets. On existence, we show an improvement over base policy performance in 46 out of 51 tasks across five simulated manipulation benchmarks, and 4 real-world tasks with a variety of pretrained policy classes. When we apply our approach on real robot hardware, we increase the relative success rate of an object picking task by 18%, and a piston assembly task by 28%, with the ticket search taking less than 150 episodes per task. On sample-efficiency, we find

that golden tickets achieve comparable sample efficiency to that of DSRL, especially with larger action spaces such as of VLAs, without requiring extensive hyperparameter tuning nor forcing the policy to tile its noise space. Finally, we also show that our method can be used for multi-reward optimization without incurring additional samples. Our contributions are:

- Golden Ticket Search, a derivative-free algorithm for finding golden tickets via Monte-Carlo policy evaluation.
- Evidence of the existence, prevalence, and empirical performance of golden tickets across model architectures, observation modalities, and embodiments in 51 real and simulated tasks.
- Demonstration that the search for golden tickets can optimize multiple downstream reward functions (i.e., success and speed) without additional environment interaction.
- Open-source code for GT Search and identified golden tickets for pretrained VLA, diffusion, and flow-matching policies in three simulated manipulation benchmarks.

II. BACKGROUND

A. Markov Decision Processes.

We model robot manipulation as a Markov Decision Process (MDP). An MDP comprises a tuple $\{S, A, T, R, p_0, \gamma\}$, where S and A are sets of states and actions (respectively), $T(s' | s, a) = \Pr(s' | s, a)$ is the transition dynamics, $R(s, a)$ is the reward function, $p_0 \in \Delta_S$ is the initial state distribution over S , and γ is the discount factor. A policy π rolls out episodes from an initial state $s_0 \sim p_0$ by iteratively sampling an action $a_t = \pi(s_t)$ to execute at each time step t to get the next state $s_{t+1} \sim T(s_t, a_t)$ and reward $r_t = R(s_t, a_t)$. The Q -function $Q^\pi(s, a)$ for π gives the expected discounted return of π rolled out from a state s with initial action a , following π for all subsequent actions. That is, $Q^\pi(s, a) = \mathbb{E}_{P^\pi, T}[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) | s_0 = s, a_0 = a]$.

B. Diffusion and Flow Matching.

These generative methods iteratively refine a sample from a source distribution into a desired target data distribution. We focus on the flow matching framework with a Gaussian

source distribution for illustration. In flow matching, given a dataset $D = \{s_t, a_t \sim p_{\text{data}}\}_{t=1}^n$, the forward process gradually corrupts the action $a \in D$ over time τ ¹ by linearly interpolating it with a sample from an isotropic Gaussian: $a^{(\tau)} = (1 - \tau)a^{(0)} + \tau w$, where $w \sim \mathcal{N}(0, I)$ and $a^{(\tau)}$ represents the partially noised action $a^{(0)}$ at time τ . Generating new samples from the data distribution p_{data} requires reversing the forward process. Specifically, a denoising model $\hat{u} = \hat{u}(s, a^{(\tau)}, \tau)$ is trained to undo the forward process, and new samples from p_{data} can be generated by iteratively denoising a sample $a^{(\tau)}$ with $a^{(1)} \sim \mathcal{N}(0, I)$, treating the denoising model $\hat{u}$ as a velocity field and solving the ODE $\hat{a}^{(0)} = a^{(1)} + \int_1^0 \hat{u}(s, a^{(\tau)}, \tau) d\tau$. While diffusion and other flow variants differ in how they define the forward and reverse processes, our method is agnostic to these variations; it focuses only on making $a^{(1)}$, the initial noise vector used for inference, a constant vector rather than repeatedly sampling it from a Gaussian distribution. We consider DDIM sampling for diffusion models due to more efficient generation [20], which is critical in robotics.

III. RELATED WORK

A. Lottery Ticket Hypothesis for Diffusion/Flow Models.

The computer vision community has also evidenced this phenomenon, named “lottery ticket hypothesis” [16]: “randomly initialized Gaussian noise images contain special pixel blocks (winning tickets) that naturally tend to be denoised into specific content independently.” Followup works have investigated how these “winning tickets” (also referred to as “golden noises” [17]–[19], or “golden tickets” in our work) can be optimized to achieve higher text-image alignment and higher human preference when they are used instead of sampling from isotropic Gaussian noise. However, finding golden tickets for robot control policies present additional difficulties due to the temporal nature of the problem and dimensionality of the problem space. The novelty of this work lies in empirically demonstrating the prevalence of golden tickets for robot policies and formulating their search as an episodic derivative-free policy improvement method.

B. Policy Improvement via Latent Steering.

While there is a diverse range of policy improvement methods, we focus on those that use latent steering, since they most closely relate to our approach. Diffusion Steering via Reinforcement Learning (DSRL) [4] swaps the source noise distribution with an observation-conditioned noise policy trained via reinforcement learning (RL). However, DSRL presents two major challenges in practice (see Fig. 1): 1) it trains actor and 2–10 critic networks, which involves modeling decisions to ensure sufficient model capacity, and 2) it integrates with an RL framework to train the noise policy, which requires hyperparameter tuning [4]. Because

golden tickets are akin to a noise policy that is observation-independent (unlike DSRL), they require no additional trainable network beyond the original policy and fewer hyperparameter adjustments. Further, DSRL fails when directly used with larger action spaces of VLAs as shown in Fig. 1. DSRL instead tiles the noise space, which departs from the i.i.d. pretraining noise and removes all temporal variation within the chunk [6]. GT Search works directly in high dimensional noise spaces of VLAs without any such forced tiling.

C. Policy Search and Bandit-Based Evaluation

Our search methods belong to the long line of direct policy search, which treats policy performance as a black-box objective and optimizes it from rollout returns alone, without gradients through the policy or the environment [10], [14], [15]. Step-based methods perturb the action at every timestep and estimate a per-step gradient, often via the likelihood-ratio estimator [21]. The variance of these estimates grows with both the horizon and the dimension of the action space. Episode-based methods instead draw one perturbation per rollout in parameter space (the latent-noise space for GT) and score it by return, using finite-difference or smoothing estimators [14], [15] or rank-based selection [12], [13]. Evolution strategies trade sample efficiency for near-linear parallelism [14], while ARS [15] shows that random search over simple policy classes matches gradient-based deep RL on standard continuous-control benchmarks. Our work continues this line of inquiry, but searches the latent input to a frozen generative policy rather than the parameters of an actor. GT Search also benefits from pure-exploration bandit [22] methods. We show that sequential halving [23] improves the sample-efficiency of CEM, eliminating poor tickets cheaply and concentrating evaluation on contenders.

IV. METHODOLOGY

A. Effect of Constant Initial Noise on Policy Rollouts

Randomness in generative policies can emanate from the initial noise or the sampling process used: DDPM [1] for stochastic sampling, and DDIM [20] for deterministic sampling. DDIM has gradually evolved as the standard choice due to more efficient sampling [4], where the initial noise remains the sole source of stochasticity in the policy. To understand the effect of constant initial noise on temporal policy rollouts and how it can serve as an instrument to optimize the downstream reward, we use the Avoiding-M2 task (Fig. 3) from D3IL [24]. This task requires the robot to reach the finish line without colliding with three levels of obstacles. As visualized in Fig. 3(b), the learned policy can often exhibit unwanted or misaligned behaviors (*gray or green*). Fig. 3(c) shows the rollouts of 300 constant initial noise vectors, or lottery tickets. Initial noise vectors provide a broad mode coverage for us to choose a ticket that optimizes the downstream reward. While several tickets avoid L11, only a few go on to also avoid L32 to be counted as *red*, and even fewer form the *green* mode, which lacks support in the demonstration data itself. Hence, GTs depend on a combination of factors such as the support induced by the learned

¹We use a superscript (τ) for the forward/reverse process time to distinguish it from the MDP time subscript t .

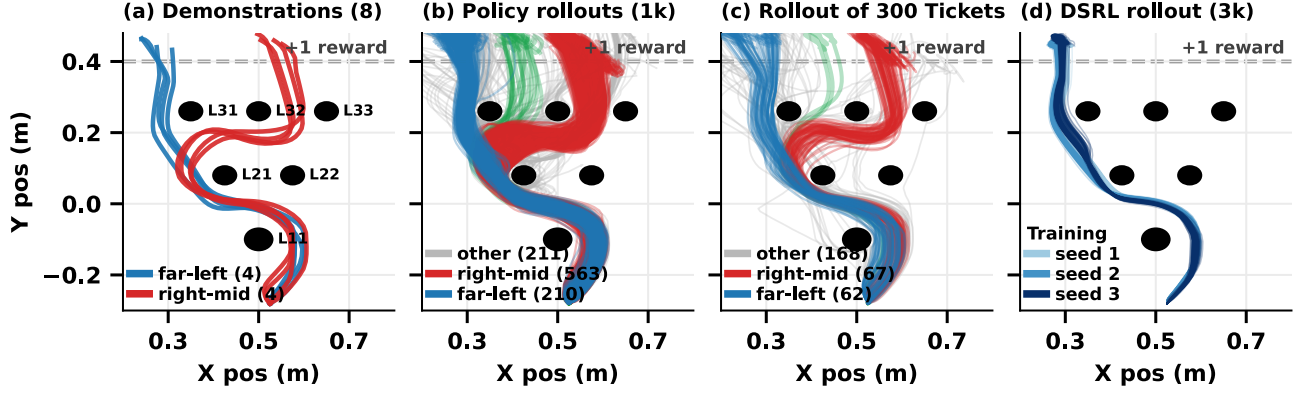


Fig. 3: D3IL task *Avoiding-M2*: (a) Demonstrations the policy was trained on covering blue and red modes; (b) Learned policies often generate diverse behaviors (gray and green); (c) Initial noises (300 here) provide a broad mode coverage for us to choose a noise sample that optimizes the downstream reward; (d) DSRL, optimized for the blue mode. See Section IV-A.

policy and the downstream reward. Interestingly, depending on the underlying search method used, once noise vectors and the corresponding rollouts are collected, they can be re-scored with different objectives (such as the green mode) to identify tickets optimizing a different or a combination of reward functions, without incurring additional environment samples. Finally, Fig. 3(d) shows rollouts from a DSRL-augmented policy having converged to the blue mode. Later optimizing the policy for the red mode will require a new DSRL runs incurring additional environment samples, unlike random search for golden tickets.

B. The Lottery Ticket Hypothesis for Robot Control.

Our main contribution is framing the search for a golden ticket, a fixed initial noise vector, as a gradient-free approach for improving policy performance on a downstream task. We assume we are given a pretrained, frozen diffusion or flow matching robot control policy π , and treat this policy as a black box without access to any intermediate steps of the denoising process. Given an MDP representing a downstream task from which we can sample episode transitions, our goal is to adapt the policy π 's behavior to maximize rewards while keeping the parameters frozen. We propose: *For a pretrained diffusion or flow matching policy and a downstream task, there exists a fixed initial noise vector w^* that, used in place of sampling from the Gaussian prior, increases the policy's expected return on held-out task evaluations.*²

We call such a noise vector a *golden ticket* and test the hypothesis empirically in Section V-B.1. Searching for a golden ticket on a robot policy raises several challenges: (1) the policy acts in an environment where current actions affect future states, (2) collecting samples from that environment is costly, and (3) action and noise space of the policy may be high-dimensional. In this work, we develop a black-box search-based method using Monte-Carlo policy evaluation to address these challenges.

²Empirically, this holds for 46 of the 51 tasks in our benchmarks for a fixed number of trials.

C. Golden Tickets Search

We propose to use black-box *search* for finding golden tickets w^* with Monte-Carlo estimates of episodic-reward obtained from policy rollouts as the objective. Formally, let $\pi_w(s)$ denote a policy whose denoising process is always initialized with noise $w \in \mathbb{R}^{H \times d_a}$, where H is the horizon and d_a is the per-step action dimension; the base policy $\pi(s)$ samples a fresh $w \sim \mathcal{N}(0, I)$ at every action step. We seek w^* that maximizes the cumulative discounted reward obtained by π_w , averaged over M rollouts from fixed initial states $\{s_0^{(m)}\}_{m=1}^M$: that is, $w^* = \operatorname{argmax}_w \frac{1}{M} \sum_{m=1}^M \sum_{t \geq 0} \gamma^t R(s_t^{(m)}, \pi_w(s_t^{(m)}))$.

Algorithm 1 CEM for Optimizing Initial Noise

- 1: **Input:** policy π , environments $\mathcal{M}$, population P , iterations T , elite fraction ρ , smoothing α , std floor $\sigma_{\min}$, extra std σ_{ex}
- 2: $\mu \leftarrow 0, \sigma \leftarrow 1, K \leftarrow \max(1, \lfloor \rho P \rfloor), \bar{R}_{\text{best}} \leftarrow -\infty$
- 3: **for** $t = 1$ to T **do**
- 4: $\epsilon_i \leftarrow \mu + \sigma \odot z_i, z_i \sim \mathcal{N}(0, I), i = 1, \dots, P$
- 5: $\bar{R}_i \leftarrow \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} R_m(\pi, \epsilon_i)$ ▷ roll out π with fixed noise ϵ_i
- 6: $\epsilon^* \leftarrow \epsilon_{\arg \max_i \bar{R}_i}$ if $\max_i \bar{R}_i > \bar{R}_{\text{best}}$; update $\bar{R}_{\text{best}}$
- 7: $\mathcal{E} \leftarrow$ top- K indices by $\bar{R}_i$
- 8: $\mu \leftarrow \alpha \operatorname{mean}(\epsilon_{\mathcal{E}}) + (1 - \alpha) \mu$
- 9: $\sigma \leftarrow \max(\alpha \operatorname{std}(\epsilon_{\mathcal{E}}) + (1 - \alpha) \sigma, \sigma_{\min})$
- 10: $\sigma \leftarrow \sqrt{\sigma^2 + \sigma_{\text{ex}}^2}$ ▷ if $\sigma_{\text{ex}} > 0$
- 11: **return** ϵ^*

In this work, we consider random search (RS) [11] and the cross-entropy method (CEM) [12] for finding golden tickets. The simplest of them, random search, samples N tickets and returns the one with the maximum average reward. CEM [12], shown in Algorithm 1, iteratively refines a Gaussian sampling distribution $\mathcal{N}(\mu, \operatorname{diag}(\sigma^2))$ over noise vectors, initialized at $\mu = 0, \sigma = 1$. Further, with sequential halving [23], the environment budget $|\mathcal{M}|$ is partitioned into R disjoint rounds of $M_0 = |\mathcal{M}|/R$ environments each.

Unlike uniform allocation, a candidate surviving every round accumulates $R \cdot M_0 = |\mathcal{M}|$ environment evaluations, while a candidate eliminated at round r contributes only $r \cdot M_0$. While both RS and CEM only rank noise vectors, zeroth-order search (ZOS) [11] instead estimates a per-coordinate gradient from scalar returns, whose variance scales as $O(Hd_a)$ [25]; we observe ZOS failing on some seeds.

Evaluating Golden Tickets. To determine if the returned ticket is “golden” (i.e., that it has higher average performance than sampling from the Gaussian prior), we assume there is a held-out set of initial states or environments which we use to evaluate the performance of both the best ticket and the base policy. For a fixed compute budget, there exists a trade-off between number of tickets N and number of search environments M on which to evaluate those tickets. More tickets result in better coverage and potentially better performance, but risk not generalizing to held-out environments.

V. EXPERIMENTS

Our experimental evaluation is designed to assess the lottery ticket hypothesis for robot control policies and provide empirical evidence of its effectiveness and practical utility.

A. Experimental Benchmarks

We investigate a wide range of model architectures, observation inputs, and environments. On real Franka hardware (Fig. 4), we evaluate two RGB diffusion policies (piston assembly and cube picking) and two point-cloud diffusion policies (banana picking and cup pushing); all hardware experiments optimize a binary end-of-episode success signal. We also report results on the GR00T N1.5 [26] public checkpoint for 7 tasks of the **SimplerEnv** (WidowX) [27] environment, designed to reflect real world performance. Additionally, we consider 4 simulation benchmarks:

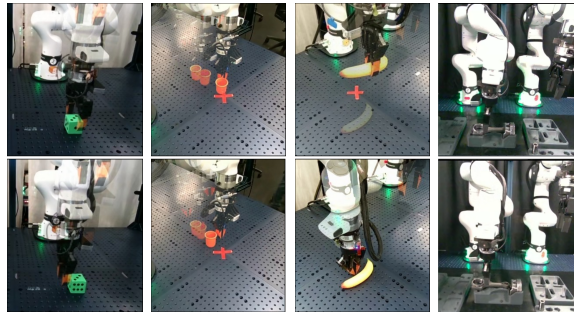
- **franka_sim** [28]: cube picking, flow-matching policy on low-dim state.
- **LIBERO** [29]: 30 tasks (Spatial/Goal/Object) with the public SmoVLA [30] checkpoint and 3 tasks from LIBERO-90 using the public π_0 checkpoint.
- **robomimic** [31]: 4 tasks with pretrained diffusion policies from DPPO [3] on low-dim state.
- **DexMimicGen** [32]: 5 bimanual tasks with RGB diffusion policies we train via BC.

In total, the suite covers 51 tasks and spans flow matching, BC-trained diffusion and VLA policies, with state, RGB, point-cloud, and language inputs across both simulation and real hardware. Additional benchmark details are given in Appendix II.

B. Experimental Results

1) Do golden tickets for robot control policies exist, and how prevalent are they? How much do they improve performance compared to sampling Gaussian noise?:

We evaluate the existence and prevalence of golden tickets across the 51 task-policy combinations in our suite. We run a large-scale search for golden tickets, with search budgets ranging from less than 250 tickets with 5 search



(a) Pick cube (b) Push cup (c) Banana (d) Piston

Fig. 4: Rollouts from diffusion policies sampling with Gaussian noise that we use in our hardware experiments. (top) An example successful rollout; (bottom) A failed rollout.

environments each in SimplerEnv to 5000 tickets per task with 100 search environments each in robomimic. We find that *golden tickets outperform Gaussian noise in 46 out of 51 tasks, and at least match it in 49* (Figure 5). Under the null hypothesis that a searched noise vector outperforms Gaussian sampling no more often than chance ($H_0: p \leq 0.5$), the observed proportion is highly significant: one-sided binomial p-value $\approx 1.2 \times 10^{-9}$, with Wilson 95% CI for p of $[0.79, 0.96]$. In franka_sim cube picking, the base policy averages 38.5% success and the best golden tickets average 96%. In robomimic, golden tickets are found in 3 of 4 tasks (not Square), with the most extreme gain on Can, from 42.8% to 95.0%. In LIBERO, evaluating per-task golden tickets, we obtain +13.4%, +12.8%, and +8% improvements over the base policy on Spatial, Goal, and Object, respectively. In DexMimicGen, golden tickets are found in 4 of 5 tasks; the biggest improvement is on Box Cleanup (87.6% $\rightarrow$ 97.8%), while Threading shows base 62% vs. best ticket 60%. In SimplerEnv (WidowX), the average improvement is +30.4% with the largest gain on put_eggplant_in_sink (21% $\rightarrow$ 100%). On real Franka hardware, the average improvement is +31%. Beyond average improvement, we examine when golden tickets reach a high *absolute* success rate: among the 34 task-policy pairs with base success rate $\geq 71\%$, golden tickets achieve $\geq 90\%$ success in 31 of them; among the 15 pairs with base $\geq 87.6\%$, they achieve $\geq 97\%$ in 14 of them. This suggests that golden tickets present a meaningful path to improving already high-performing behavior cloning policies. The significance test and the reach-threshold computation are detailed in Appendices IV and V.

2) How competitive is searching for golden tickets to state-of-the-art latent steering and reinforcement learning methods?: We compare random search and CEM (Algorithm 1) against six gradient-based RL baselines including DSRL, with tuned implementations for a state-based robomimic task Can (Fig. 6a; see [3] for baseline details). The other five gradient-based baselines either adapt the pretrained policy behavior (DPPO [3], IDQL [33], DQL [34]), or use diffusion but learn from scratch (DIPO [35], QSM [36]). On Can, merely searching for a good fixed initial noise is competitive with gradient-based adaptation methods, even for MLP diffusion policies trained in state-based tasks

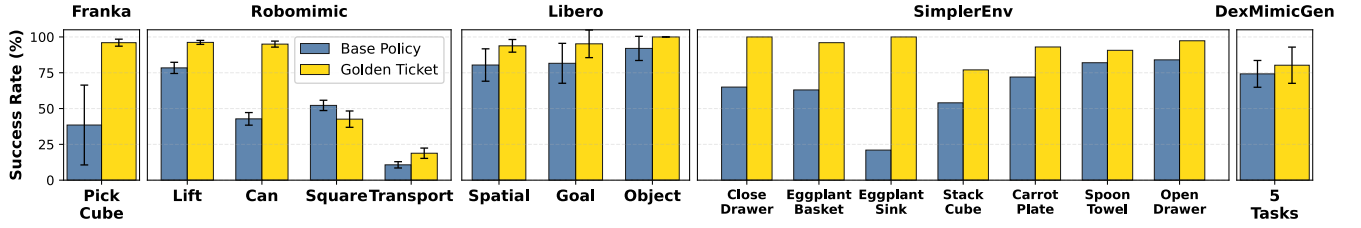


Fig. 5: Existence and performance of golden tickets (gold, right) relative to the base policy (blue, left) on simulated benchmarks. Simply optimizing the initial noise vector can lead to strong performance gains over the base policy across benchmarks and policies. We report mean and standard deviation of success rates (details in Section V-B). Our open-source repository will contain code and golden tickets for the first three benchmarks: franka_sim, robomimic, and LIBERO.

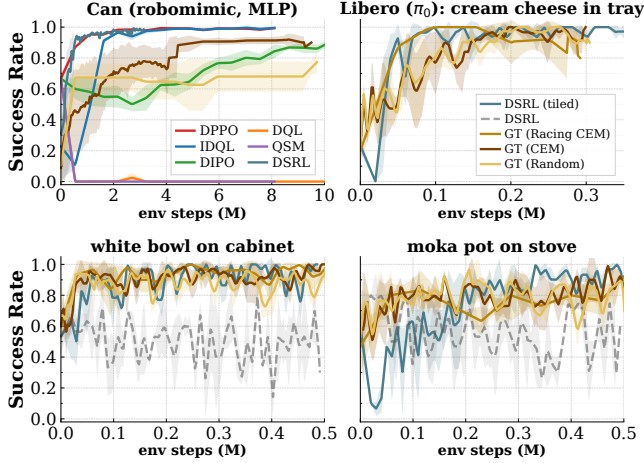


Fig. 6: Sample efficiency comparison on Can with an MLP base policy (a) and three LIBERO-90 tasks with π_0 (b–d). Simply searching for constant noise vectors (GT) becomes comparable in sample efficiency to actor-critic methods at the high-dimensional noise spaces of VLA models, where DSRL on the full 1600D noise (dashed) fails to improve on the base policy.

– outperforming QSM, DQL, and IDQL.

With d_a as the action dimension and H as the prediction horizon, as the noise space grows from $d_a \times H = 7 \times 4 = 28\text{D}$ in Can MLP to $32 \times 50 = 1600\text{D}$ in LIBERO π_0 , we see DSRL falter (Fig. 6). DSRL updates its actor along $\nabla_w Q$ of a critic fit by temporal-difference regression over the $d_a H$ -dimensional noise space. As $d_a H$ grows, the critic’s data covers a smaller fraction of this space [37], and the actor’s ascent moves toward directions where Q is overestimated, which the critic then bootstraps from [38]. DSRL tiles its noise space to reduce this regression to d_a dimensions; GT search needs no such restriction.

Surprisingly, we see in Figure 6b,c,d that naive random sampling of constant initial noises from the 1600D noise space leads to comparable performance as running an actor-critic in the tiled noise space. Random search for golden tickets depends on the volume fraction of tickets in the noise space of the policy that cross the expected reward threshold. This does not directly depend on the action-space dimensionality, which is why we see GT Search perform well even for larger action spaces, such as those of VLAs.

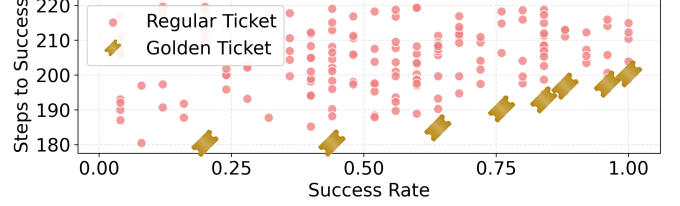


Fig. 7: Various tickets (pink) for the franka_sim pick policy, evaluated according to success rate and speed (mean length of successful episodes). Right is higher success rate, down is faster time to success. Because lottery tickets exhibit extreme differences in policy performance, a Pareto frontier is defined by tickets that are further right/down than others (represented with golden ticket icons).

For example, we find that only 2.5% of the sampled 200 tickets resulted in greater than base policy performance for 28D Can, while over 50% of the sampled tickets of 200 were better than the base policy for LIBERO#57 π_0 with a 1600D action space. Golden tickets benefit from being an episodic policy-improvement algorithm [15]: they do not need to ascend along the coordinates of the noise space, which is what hurts DSRL as $d_a H$ grows.

3) *Do lottery tickets define a Pareto frontier when given multiple objectives to optimize?*: We investigate this in franka_sim with a block picking task under two objectives—maximizing success rate, and maximizing speed of successful episodes—which trade off against each other (the faster the robot moves, the harder it becomes to successfully pick up the block). We run 400 tickets in 50 different start states each, and evaluate each ticket on both objectives. *Golden tickets are sufficiently varied to define a Pareto frontier that balances success rate and speed for an object picking task* (Fig. 7). Ticket success rates vary from $\approx 5\%$ to nearly 100%, and speeds of successful episodes range from ≈ 180 to 220 steps; this diversity results in a small subset of golden tickets that define a Pareto frontier, outperforming all other regular tickets to the left and above them and representing a balance between the two objectives. This suggests a simple way to alter a robot’s behavior to adapt between objectives online: collect the golden tickets on the Pareto frontier, and switch between them as desired.

4) *How effective is Golden Tickets Search in practice and how do they perform on hardware?*: On hardware, Golden Ticket Search delivers substantial gains within 1 hour

searching for a constant initial noise vector can lead to comparable performance to sophisticated gradient-based methods requiring cumbersome hyperparameter tuning.

- Latent steering is bounded by the support of the base policy: a golden ticket can only elicit behaviors that the frozen policy is already capable of producing.
- Finally, as with other latent policy steering approaches [7], GT Search chooses high reward modes for improving the performance, while also resulting in deterministic rollouts.

VIII. CONCLUSION

Prior work on generative policy improvement has largely focused on derivative-based approaches. We propose an episodic, derivative-free method based on searching for constant initial noises in the policy’s latent space. Our approach to improving pretrained diffusion or flow-matching policies avoids 1) adjusting the original policy weights, 2) training additional neural networks, and 3) restricting the policy’s noise space, as tiling does. Through experiments, we demonstrate the prevalence of golden tickets and show that simply optimizing these noise vectors can match state-of-the-art derivative-based latent policy improvement methods, especially in high-dimensional action spaces. We also release an open-source codebase with models, environments, and golden tickets for VLAs, diffusion policies, and flow-matching policies. <https://lottery-tickets.rai-inst.com/>

ACKNOWLEDGMENT

We thank Stefanie Tellex for providing valuable feedback on the manuscript.

REFERENCES

- [1] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” 2020. [Online]. Available: <https://arxiv.org/abs/2006.11239>
- [2] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” 2023. [Online]. Available: <https://arxiv.org/abs/2210.02747>
- [3] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz, “Diffusion policy optimization,” *arXiv preprint arXiv:2409.00588*, 2024.
- [4] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, “Steering your diffusion policy with latent space reinforcement learning,” *arXiv preprint arXiv:2506.15799*, 2025.
- [5] Y. Wang, L. Wang, Y. Du, B. Sundaralingam, X. Yang, Y.-W. Chao, C. Perez-D’Arpino, D. Fox, and J. Shah, “Inference-time policy steering through human interactions,” 2025. [Online]. Available: <https://arxiv.org/abs/2411.16627>
- [6] L. Gu, C. Hou, M. Li, X. Wang, J. Liu, X. Lv, B. Zhang, S. Bai, G. Li, J. He *et al.*, “Haf: Adapting generalist vlars to humanoid whole-body loco-manipulation via hierarchical action flow and spectral latent rl,” *arXiv preprint arXiv:2608.16837*, 2026.
- [7] A. Longhini, D. Emukpere, J.-M. Renders, and S. Kim, “Behavioral mode discovery for fine-tuning multimodal generative policies,” *arXiv preprint arXiv:2605.11387*, 2026.
- [8] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, “Residual policy learning,” *arXiv preprint arXiv:1812.06298*, 2018.
- [9] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, D. Driess, M. Equi, A. Esmail, Y. Fang, C. Finn, C. Glossop, T. Godden, I. Goryachev, L. Groom, H. Hancock, K. Hausman, G. Hussein, B. Ichter, S. Jakubczak, R. Jen, T. Jones, B. Katz, L. Ke, C. Kuchi, M. Lamb, D. LeBlanc, S. Levine, A. Li-Bell, Y. Lu, V. Mano, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, C. Sharma, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, W. Stoeckle, A. Swerdlow, J. Tanner, M. Torne, Q. Vuong, A. Walling, H. Wang, B. Williams, S. Yoo, L. Yu, U. Zhilinsky, and Z. Zhou, “ $\pi_{0.6}^*$: a vla that learns from experience,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.14759>
- [10] M. P. Deisenroth, G. Neumann, and J. Peters, “A survey on policy search for robotics,” *Foundations and Trends® in Robotics*, vol. 2, no. 1-2, pp. 1–142, 2013.
- [11] A. Jordana, J. Zhang, J. Amigo, and L. Righetti, “An introduction to zero-order optimization techniques for robotics,” *arXiv preprint arXiv:2506.22087*, 2025.
- [12] R. Y. Rubinstein, “Optimization of computer simulation models with rare events,” *European Journal of Operational Research*, vol. 99, no. 1, pp. 89–112, 1997.
- [13] N. Hansen and A. Ostermeier, “Completely derandomized self-adaptation in evolution strategies,” *Evolutionary computation*, vol. 9, no. 2, pp. 159–195, 2001.
- [14] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, “Evolution strategies as a scalable alternative to reinforcement learning,” *arXiv preprint arXiv:1703.03864*, 2017.
- [15] H. Mania, A. Guy, and B. Recht, “Simple random search provides a competitive approach to reinforcement learning,” *arXiv preprint arXiv:1803.07055*, 2018.
- [16] J. Mao, X. Wang, and K. Aizawa, “The lottery ticket hypothesis in denoising: Towards semantic-driven initialization,” in *European Conference on Computer Vision*. Springer, 2024, pp. 93–109.
- [17] Z. Zhou, S. Shao, L. Bai, S. Zhang, Z. Xu, B. Han, and Z. Xie, “Golden noise for diffusion models: A learning framework,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 17 688–17 697.
- [18] C. Chen, L. Yang, X. Yang, L. Chen, G. He, C. Wang, and Y. Li, “Find: Fine-tuning initial noise distribution with policy optimization for diffusion models,” in *Proceedings of the 32nd ACM International Conference on Multimedia*, 2024, pp. 6735–6744.
- [19] Y. Miao, W. Loh, P. Poupart, and S. Kothawade, “A minimalist method for fine-tuning text-to-image diffusion models,” *arXiv preprint arXiv:2506.12036*, 2025.
- [20] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” 2022. [Online]. Available: <https://arxiv.org/abs/2010.02502>
- [21] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine learning*, vol. 8, no. 3, pp. 229–256, 1992.
- [22] S. Bubeck, R. Munos, and G. Stoltz, “Pure exploration in finitely-armed and continuous-armed bandits,” *Theoretical Computer Science*, vol. 412, no. 19, pp. 1832–1852, 2011.
- [23] Z. Karnin, T. Koren, and O. Somekh, “Almost optimal exploration in multi-armed bandits,” in *International conference on machine learning*. PMLR, 2013, pp. 1238–1246.
- [24] X. Jia, D. Blessing, X. Jiang, M. Reuss, A. Donat, R. Lioutikov, and G. Neumann, “Towards diverse behaviors: A benchmark for imitation learning with human demonstrations,” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 41 666–41 686.
- [25] J. C. Duchi, M. I. Jordan, M. J. Wainwright, and A. Wibisono, “Optimal rates for zero-order convex optimization: The power of two function evaluations,” *IEEE Transactions on Information Theory*, vol. 61, no. 5, pp. 2788–2806, 2015.
- [26] J. Björck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang *et al.*, “Gr00t n1: An open foundation model for generalist humanoid robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [27] X. Li, K. Hsu, J. Gu, K. Pertsch, O. Mees, H. R. Walke, C. Fu, I. Lunawat, I. Sieh, S. Kirmani, S. Levine, J. Wu, C. Finn, H. Su, Q. Vuong, and T. Xiao, “Evaluating real-world robot manipulation policies in simulation,” *arXiv preprint arXiv:2405.05941*, 2024.
- [28] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine, “Serl: A software suite for sample-efficient robotic reinforcement learning,” 2024.
- [29] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Liber0: Benchmarking knowledge transfer for lifelong robot learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.
- [30] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zoutine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti *et al.*, “Smolvla: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [31] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters

- in learning from offline human demonstrations for robot manipulation,” *arXiv preprint arXiv:2108.03298*, 2021.
- [32] Z. Jiang, Y. Xie, K. Lin, Z. Xu, W. Wan, A. Mandlekar, L. J. Fan, and Y. Zhu, “Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 16 923–16 930.
 - [33] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine, “Idql: Implicit q-learning as an actor-critic method with diffusion policies,” *arXiv preprint arXiv:2304.10573*, 2023.
 - [34] Z. Wang, J. J. Hunt, and M. Zhou, “Diffusion policies as an expressive policy class for offline reinforcement learning,” *arXiv preprint arXiv:2208.06193*, 2022.
 - [35] L. Yang, Z. Huang, F. Lei, Y. Zhong, Y. Yang, C. Fang, S. Wen, B. Zhou, and Z. Lin, “Policy representation via diffusion probability model for reinforcement learning,” *arXiv preprint arXiv:2305.13122*, 2023.
 - [36] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma, “Learning a diffusion model policy from rewards via q-score matching,” *arXiv preprint arXiv:2312.11752*, 2023.
 - [37] R. Kleinberg, A. Slivkins, and E. Upfal, “Multi-armed bandits in metric spaces,” in *Proceedings of the fortieth annual ACM symposium on Theory of computing*, 2008, pp. 681–690.
 - [38] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 1587–1596. [Online]. Available: <https://proceedings.mlr.press/v80/fujimoto18a.html>
 - [39] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *CoRR*, vol. abs/2106.09685, 2021. [Online]. Available: <https://arxiv.org/abs/2106.09685>
 - [40] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, p. 02783649241273668, 2023.
 - [41] G. Lu, W. Guo, C. Zhang, Y. Zhou, H. Jiang, Z. Gao, Y. Tang, and Z. Wang, “Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning,” *arXiv preprint arXiv:2505.18719*, 2025.
 - [42] Y. Luo, Z. Yang, F. Meng, Y. Li, J. Zhou, and Y. Zhang, “An empirical study of catastrophic forgetting in large language models during continual fine-tuning,” 2025. [Online]. Available: <https://arxiv.org/abs/2308.08747>
 - [43] M. Tan and Q. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” in *International conference on machine learning*. PMLR, 2019, pp. 6105–6114.
 - [44] M. Du and S. Song, “Dynaguide: Steering diffusion policies with active dynamic guidance,” *arXiv preprint arXiv:2506.13922*, 2025.
 - [45] M. Nakamoto, O. Mees, A. Kumar, and S. Levine, “Steering your generalists: Improving robotic foundation models via value guidance,” *arXiv preprint arXiv:2410.13816*, 2024.
 - [46] D. Hendrycks, “Gaussian error linear units (gelus),” *arXiv preprint arXiv:1606.08415*, 2016.
 - [47] D. P. Kingma, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
 - [48] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5–9, 2015, proceedings, part III 18*. Springer, 2015, pp. 234–241.
 - [49] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “Pointnet: Deep learning on point sets for 3d classification and segmentation,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 652–660.

APPENDIX I

RELATED WORK: ROBOT POLICY IMPROVEMENT METHODS

One class of approaches to improving pretrained policies involves directly finetuning the model parameters (or applying model adaption methods like Low-Rank Adaptation

(LoRA) [39]), using either supervised [40] or reinforcement learning (RL) losses [41]. These approaches apply the same pretraining techniques to improve policies, but can be challenging to use, since adjusting policy weights can lead to catastrophic forgetting [42] and is computationally hard for large models like VLAs. While some of these methods focus on RL training of auto-regressive models, more directly related to our work is DPPO [3], which backpropagates PPO gradients through the diffusion sampling process by treating an environment episode as an MDP that interleaves denoising and environment steps.

Another class of approaches learns a new model that adjusts policy output, such as residual policy learning [8] and latent steering [4]. These methods avoid adjusting pretrained policy weights, which helps preserve the original behavior, but require carefully designing/training new neural modules with sufficient model capacity to avoid acting as bottlenecks [43]. A last class of approaches involves making additional training/inference-time assumptions (e.g., inference-time steering [5], classifier-free guidance [9], world models [44], value-guidance [45]). These methods avoid adjusting the pretrained policy weights or learning additional neural modules for policy improvement, but they do not apply to all pretrained models (e.g., classifier-free guidance assumes a particular training regime) or assume extra models are available (e.g., access to a world model to use dynamics for guidance, or a user to provide corrective behaviors).

APPENDIX II

ADDITIONAL DETAILS ON BENCHMARKS

Table II summarizes the benchmark suite: the policy or public checkpoint evaluated on each benchmark, its observation and action spaces, and the tasks used. “Ours” indicates a policy trained internally. We further describe the policies trained in our work and our real robot setup below. Figure 9 shows sample images from the simulated benchmarks.

1) *Flow matching policy in franka_sim*: We use the franka_sim MuJoCo environment that was originally released in [28]. The task involves a single-arm Franka robot picking a cube; the cube spawns randomly within bounds of $[-0.25, 0.25]$ m (x) and $[0.25, 0.55]$ m (y) on the ground in front of the robot (roughly a $\frac{1}{2}$ m² region), and the goal is to lift it off the ground. We train a flow matching policy that takes as input the low-dimensional state of the cube and robot. The policy is a 4-layer MLP with GELU activations [46] and 256 hidden units per layer, and outputs an 8-step action chunk where each step contains a 3D end-effector position and a 1D gripper state (total chunk dimension $8 \times 4 = 32$). We collect 1000 demonstrations using a task-and-motion planning heuristic, and train 4 model checkpoints for 100 epochs with a batch size of 20, using the Adam optimizer [47] with a learning rate of 0.001.

2) *RGB diffusion policy in DexMimicGen*: We use the DexMimicGen benchmark [32], which involves five bimanual tasks with Franka arms: Drawer Cleanup, Tray Lift, Threading, Box Cleanup, and Piece Assembly. For each of the 5 tasks we train a separate

TABLE II: Summary of experimental benchmarks. “Ours” indicates a policy trained internally. EE = end-effector, proprio = proprioception, PCD = point cloud.

Benchmark	Policy / checkpoint	Obs / Action	Tasks
franka_sim [28] robomimic [31]	Flow-matching MLP (open-sourced) DPPO diffusion github.com/irom-princeton/dppo	state / EE-pos + gripper state / EE-pose	pick_cube Lift, Can, Square, Transport
LIBERO [29]	SmolVLA [30] huggingface.co/HuggingFaceVLA/ smolvla_libero	2×RGB + state + lang / 32×50 chunk	30 tasks across Object, Goal, Spatial (10 each)
SimplerEnv (WidowX) [27]	GR00T N1.5 [26] github.com/NVIDIA/Isaac-GR00T /tree/n1.5-release	RGB + lang / EE	close_drawer, open_drawer, put_eggplant_in_basket, put_eggplant_in_sink, stack_cube, carrot_on_plate, spoon_on_towel
DexMimicGen [32]	U-Net diffusion (ours)	3×RGB + proprio / bimanual EE	Drawer Cleanup, Tray Lift, Threading, Box Cleanup, Piece Assembly
Franka HW (RGB)	U-Net diffusion (ours)	3×RGB + proprio / EE	cube_pick, piston_assembly
Franka HW (PCD)	U-Net diffusion (ours)	PCD + proprio / EE	banana_pick, cup_push



Fig. 9: Images from some of our simulated benchmarks: (1-3) LIBERO Object, Spatial, and Goal task suites, (4-6) DexMimicGen Tray Lift, Threading, and Piece Assembly tasks, and (7-9) robomimic Lift, Can and Square.

diffusion policy with a U-Net backbone [48], a ResNet-18 encoder for the RGB images, and an MLP for the robot proprioception data. The policy takes in 3 RGB images and the end-effector position, quaternion, and gripper state for both arms.

3) *Franka hardware - RGB diffusion policy*: We use Franka hardware to conduct real-world experiments: the Franka Research 3 arm is equipped with a Robotiq 2F-85 gripper, with RealSense D435 cameras for the two static external cameras and a D405 for the wrist camera. We train RGB diffusion policies (one wrist camera, two static external cameras, and proprioception data) via behavior cloning on a block picking task (Figure 4a) and piston-assembly task. Both policies use a standard diffusion policy architecture with a U-Net backbone and ResNet-18 image encoders. We optimize a binary success signal as the reward function: for block picking, it is whether the cube is lifted off the table, and for piston assembly, it is based on whether the piston head and rod are properly mated for subsequent pin insertion.

4) *Franka hardware - Pointcloud diffusion policy*: We use the same Franka hardware described above, but with two pointcloud policies for two separate tasks: (1) picking a banana (Figure 4c) and (2) pushing a cup to the center of the table (Figure 4b). For the pointcloud policies, we use the same U-Net backbone as in the RGB setup but replace the image encoder with a PointNet encoder [49] for the pointcloud. We use 2 calibrated extrinsic cameras to generate a single fused pointcloud, and remove all points that are at or below the surface of the table. We again optimize a binary success signal as the reward function.

APPENDIX III

EXISTENCE OF GOLDEN TICKETS

Golden tickets outperform Gaussian noise in 46 out of 51 tasks, and at least match it in 49. Table IV lists, for every task-policy pair, the held-out success rate of the base policy and of the best golden ticket, together with the per-task search budget and search method. Overall, the results indicate that the lottery ticket hypothesis is often true, in the sense that golden tickets can likely be found that improve over the base policy. Table III breaks the SimplerEnv results down by search seed.

APPENDIX IV

STATISTICAL SIGNIFICANCE OF THE IMPROVEMENT RATE

Throughout, *base* denotes the held-out success rate of the pretrained policy with Gaussian initial noise, and *gold* the held-out success rate of the best searched constant initial noise (golden ticket) for the same task-policy pair. The suite comprises $n = 51$ pairs: franka_sim (1, mean of 4 seeds), robomimic (4), LIBERO (30), DexMimicGen (5), SimplerEnv (WidowX) (7), and real Franka hardware (4).

For each of the $n = 51$ tasks i , we define a binary outcome $X_i = 1$ if the best searched noise vector strictly outperforms the base (Gaussian noise) policy on the held-out evaluation, and $X_i = 0$ otherwise. Modeling $X_i \stackrel{iid}{\sim} \text{Bernoulli}(p)$, we test

$$H_0: p \leq 0.5 \quad \text{vs.} \quad H_1: p > 0.5,$$

where p is the probability that, for a task drawn from the distribution sampled by our benchmark suite (franka_sim $\cup$ LIBERO $\cup$ robomimic $\cup$ DexMimicGen $\cup$ SimplerEnv (WidowX) $\cup$ real-world), the best searched noise vector strictly improves over Gaussian sampling. The null $p_0 = \frac{1}{2}$ corresponds to the standard sign-test null.

TABLE III: SimplierEnv (WidowX) results over three search seeds. **Gold**: best held-out success rate (% , 300 episodes) in each seed’s CEM run; N : CEM candidates evaluated in each run.

Task	Base	Avg. Gold	Gold	N
close_drawer	65	99.2	99.0/98.7/100.0	11/16/6
put_eggplant_in_basket	63	91.7	96.0/93.3/85.7	16/32/14
put_eggplant_in_sink	21	69.3	67.7/64.0/76.3	270/27/100
stack_cube	54	71.3	71.7/73.7/68.7	230/69/27
carrot_on_plate	72	74.3	78.7/51.3/93.0	150/106/81
spoon_on_towel	82	79.9	77.3/72.3/90.0	230/53/66
open_drawer	84	95.7	96.0/97.3/93.7	183/121/144

Letting $K = \sum_i X_i$, the exact one-sided p-value is $\mathbb{P}_{p=0.5}(K \geq k_{\text{obs}}) = 2^{-n} \sum_{j=k_{\text{obs}}}^n \binom{n}{j}$. For $k = 46$ we obtain p-value $\approx 1.16 \times 10^{-9}$, and for the weaker “at-least-match” count $k = 49$, p-value $\approx 5.89 \times 10^{-13}$. We report a Wilson 95% score interval for p ,

$$\text{CI} = \frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + z^2/n}, \quad z = 1.96,$$

which gives $[0.79, 0.96]$ for $\hat{p} = 46/51$ and $[0.87, 0.99]$ for $\hat{p} = 49/51$.

Approximations. (i) Tasks within a benchmark may share a base policy and also the observation pipeline (e.g., all 30 LIBERO tasks use SmolVLA), so the iid assumption is approximate. (ii) The inference applies to the population sampled by our benchmarks, not to all manipulation tasks. (iii) The indicator X_i does not weight by the per-task uncertainty in the success-rate estimate.

APPENDIX V

PERFORMANCE IMPROVEMENT WITH GOLDEN TICKETS

Beyond whether golden tickets improve over Gaussian sampling (Appendix IV), we ask: *above what base policy performance do golden tickets reliably improve task success?* Concretely, we seek the lowest base policy floor T^* such that, among the task–policy pairs whose base success rate is at least T^* , at least 90% are pushed past a high absolute success bar by their golden ticket. We instantiate “high” at two levels, $G \in \{90, 97\}\%$.

For a base floor T and absolute bar G , let

$$\hat{p}(T, G) = \frac{\#\{\text{pairs with base} \geq T \text{ and gold} \geq G\}}{\#\{\text{pairs with base} \geq T\}}$$

be the empirical pass rate among the pairs included by floor T . We then report

$$T_G^* = \min\{T : \hat{p}(T', G) \geq 0.90 \text{ for every } T' \geq T\},$$

i.e., the lowest base floor above which the empirical pass rate *stays* at $\geq 90\%$. Candidate floors are the distinct base success rates in the suite. Table V reports T_G^* for both bars.

TABLE V: Base policy floor T_G^* above which golden tickets reach the absolute bar G in at least 90% of the supported task–policy pairs. n is the number of pairs with base $\geq T_G^*$; k is the number of those reaching $\geq G$.

Target G	T_G^*	Support n	Successes k	k/n
90%	71%	34	31	0.912
97%	87.6%	15	14	0.933

a) *Base floor $T^* = 71\%$ (bar $G = 90\%$, $n = 34$):*
The 34 pairs with base $\geq 71\%$ are (including 3 trivial passes where the base policy is already at 100%):

- LIBERO-Spatial (8): T0, T1, T2, T4, T6, T7, T8, T9 (excludes T3, T5)
- LIBERO-Goal (8): T0, T1, T2, T4, T5, T6, T7, T8 (excludes T3, T9)
- LIBERO-Object (10): all tasks T0–T9
- DexMimicGen (3): Drawer Cleanup, Tray Lift, Box Cleanup
- robomimic (1): Lift
- Real robot (1): cube picking
- SimplierEnv (WidowX) (3): carrot_on_plate, spoon_on_towel, open_drawer

Three of these 34 pairs do not reach $\geq 90\%$:

- DexMimicGen Tray Lift: base 71%, gold 79.9%
- DexMimicGen Drawer Cleanup: base 82.2%, gold 88%
- LIBERO-Spatial T0: base 78%, gold 86%

b) *Base floor $T^* = 87.6\%$ (bar $G = 97\%$, $n = 15$):*
The 15 pairs with base $\geq 87.6\%$ are:

- LIBERO-Spatial (3): T1, T6, T7
- LIBERO-Goal (4): T1, T4, T7, T8
- LIBERO-Object (7): T1, T2, T3, T6, T7, T8, T9
- DexMimicGen (1): Box Cleanup

One of these 15 pairs does not reach $\geq 97\%$:

- LIBERO-Spatial T7: base 90%, gold 96%

APPENDIX VI

FURTHER RESULTS

A. DexMimicGen Results

Figure 10 reports per-task success rate for DSRL and GT Search(Random Search) at two episode budgets (5k and 10k search episodes per task) crossed with two DDIM step counts (2 and 8); the dashed line in each panel marks the corresponding base-policy success rate. The five tasks are from the DexMimicGen suite: DrawerCleanup, LiftTray, Threading, BoxCleanup, and ThreePieceAssembly. GT Search appears broadly equivalent to DSRL in sample efficiency on DexMimicGen. DSRL often struggles to match base-policy

TABLE IV: Per-task summary of the golden-ticket vs. Gaussian comparison across all 51 task-policy pairs. **Base** and **Golden** are held-out success rates (%); **Search** combines the per-task budget (tickets $\times$ search-envs per ticket) with the search method (RS = random search, CEM = cross-entropy method). Shaded Avg. rows give the per-benchmark mean across the listed tasks.

Benchmark	Task	Base	Golden	Search
franka_sim	Avg. over 4 ckpts	38.5 $\pm$ 27.9	96.0 $\pm$ 2.4	$\approx 250 \times 25$ (RS)
robomimic	Lift	78.4 $\pm$ 3.9	96.2 $\pm$ 1.4	5000 $\times$ 100 (RS/CEM)
	Can	42.8 $\pm$ 4.4	95.0 $\pm$ 2.1	
	Square	52.2 $\pm$ 3.6	42.6 $\pm$ 5.7	
	Transport	10.7 $\pm$ 2.2	18.8 $\pm$ 3.6	
	Avg.	46.0	63.1	
LIBERO-Object	T0	82	100	1416 $\times$ 5 (RS)
	T1	98	100	
	T2	98	100	
	T3	98	100	
	T4	76	100	
	T5	82	100	
	T6	100	100	
	T7	90	100	
	T8	96	100	
	T9	100	100	
	Avg.	92.0	100.0	
LIBERO-Goal	T0	72	100	1338 $\times$ 3-5 (RS)
	T1	94	100	
	T2	86	100	
	T3	52	68	
	T4	92	100	
	T5	78	100	
	T6	80	94	
	T7	100	100	
	T8	94	100	
	T9	68	90	
	Avg.	81.6	95.2	
LIBERO-Spatial	T0	78	86	1081 $\times$ 3-5 (RS)
	T1	94	98	
	T2	84	94	
	T3	62	100	
	T4	84	92	
	T5	58	90	
	T6	90	100	
	T7	90	96	
	T8	78	90	
	T9	86	92	
	Avg.	80.4	93.8	
SimplerEnv (WidowX)	close_drawer	65	100.0	250 $\times$ (5-10) (CEM)
	open_drawer	84	97.3	
	put_eggplant_in_basket	63	96.0	
	put_eggplant_in_sink	21	100.0	
	stack_cube	54	77.0	
	carrot_on_plate	72	93.0	
	spoon_on_towel	82	90.7	
	Avg.	63.0	93.4	
DexMimicGen	Drawer Cleanup	82.2 $\pm$ 4.4	88.0 $\pm$ 2.9	500 $\times$ 50 (RS)
	Tray Lift	71.0 $\pm$ 3.9	79.9 $\pm$ 4.6	
	Threading	62.0 $\pm$ 4.1	60.0 $\pm$ 4.9	
	Box Cleanup	87.6 $\pm$ 2.4	97.8 $\pm$ 1.1	
	Piece Assembly	68.3 $\pm$ 3.3	75.6 $\pm$ 3.1	
	Avg.	74.2	80.3	
Franka HW	cube_pick	80	98	6 $\times$ 25 (RS)
	piston_assembly	60	88	22 tickets, 61 eps (RS)
	banana_pick	50	68	10 $\times$ 5 (RS)
	cup_push	40	100	10 $\times$ 5 (RS)
	Avg.	57.5	88.5	—

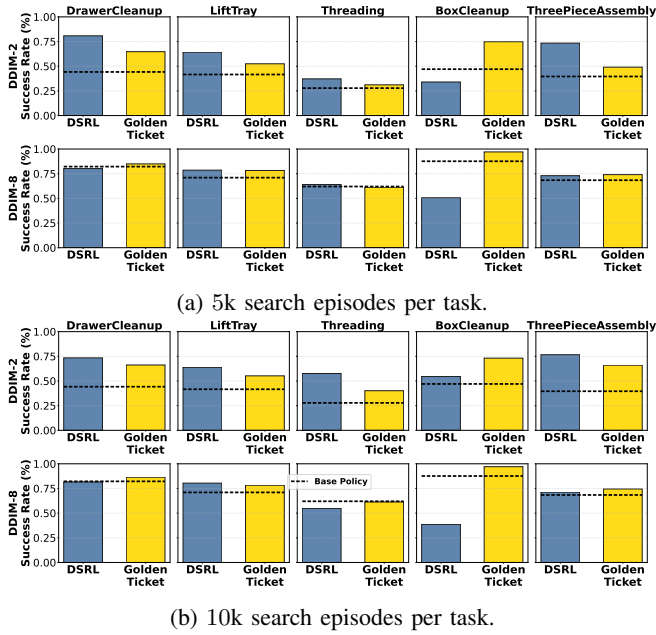


Fig. 10: DSRL (blue) vs. Golden Ticket via Random Search (yellow) on DexMimicGen across five tasks (columns), two DDIM step settings (top row of each panel: DDIM-2; bottom row: DDIM-8), and two search-episode budgets (5k in (a), 10k in (b)). The dashed horizontal line in each panel marks the base-policy success rate at that DDIM setting.

performance at DDIM-8, while doing better at DDIM-2. The standout task is BoxCleanup, where GT Search clearly outperform DSRL across DDIM steps.

B. Multi-task Results

1) *Is there evidence of golden tickets that generalize across tasks?*: In LIBERO with SmolVLA across the 3 task suites (LIBERO-Object, LIBERO-Goal, LIBERO-Spatial; 10 tasks each, 30 tasks total), we investigate whether golden tickets found (using random search) for one task can also act as golden tickets for other tasks. Across all 30 tasks, golden tickets match or exceed the base policy’s performance (Table VI). Certain golden tickets match or exceed the base policy on multiple tasks: in LIBERO-Goal, ticket #03c2 matches or exceeds the base policy in 7/10 tasks (T0, T1, T2, T4, T6, T7, T8); in LIBERO-Object, ticket #015a matches or exceeds the base policy in 7/10 tasks (T2, T3, T5, T6, T7, T8, T9); in LIBERO-Spatial, ticket #60c0 matches or exceeds the base policy in 5/10 tasks (T0, T2, T3, T6, T8). Our results provide evidence that related tasks may share golden tickets for VLA policies.

2) *Multi-task Golden Tickets Search*: We complement the per-task SimplEnv results in Appendix III with multi-task golden ticket searches on the GR00T N1.5 base policy, run separately on a drawer pair (open_drawer, close_drawer) and an eggplant pair (put_eggplant_in_sink, put_eggplant_in_basket). Each search uses CEM with population 20 for up to 25 iterations and a maximum of 500 tickets, with 10 search-environment instances per task

TABLE VI: Success rates (%) for base policy and lottery tickets on LIBERO-Goal, LIBERO-Object, and LIBERO-Spatial with SmolVLA. T0 - T9 represent different tasks. Top-1 ticket per task shown (unique set) plus best average ticket (*). Underline: $\geq$ base policy performance.

LIBERO-Goal											
#Ticket	T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	Avg
Base	72	94	86	52	92	78	80	100	94	68	81.6
#03c2 *	84	100	<u>92</u>	40	100	64	94	100	<u>98</u>	18	79.0
#2cee	78	84	100	2	98	76	22	100	86	0	64.6
#1ff8	98	64	86	14	100	0	38	96	100	2	59.8
#4d97	2	96	16	68	98	92	30	54	100	4	56.0
#672f	0	12	88	0	72	4	38	100	2	90	40.6
#0f3f	100	2	0	0	68	56	0	76	38	0	34.0
#0310	0	12	0	0	100	100	10	4	42	0	26.8

LIBERO-Object											
#Ticket	T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	Avg
Base	82	98	98	98	76	82	100	90	96	100	92.0
#015a *	36	62	100	100	16	92	100	<u>94</u>	100	100	80.0
#5159	20	86	100	100	60	100	94	36	92	100	78.8
#184f	98	0	94	86	100	0	78	100	86	44	68.6
#1944	100	38	72	94	94	4	30	90	72	0	59.4
#14e7	14	100	0	0	0	32	56	64	56	0	32.2
#096d	50	0	0	4	0	0	0	100	0	0	15.4

LIBERO-Spatial											
#Ticket	T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	Avg
Base	78	94	84	62	84	58	90	90	78	86	80.4
#60c0 *	<u>82</u>	82	<u>88</u>	100	74	24	<u>98</u>	80	<u>82</u>	68	77.8
#a68f	76	72	92	50	92	90	56	70	90	82	77.0
#ecf0	68	96	80	76	12	30	98	96	82	68	70.6
#0e0c	80	60	86	100	10	24	90	92	84	74	70.0
#05c7	70	92	88	72	52	0	100	64	44	78	66.0
#517b	72	84	92	78	2	20	66	92	62	52	62.0
#6a21	64	98	90	30	92	30	30	74	58	44	61.0
#7c0e	64	68	64	4	22	60	6	88	44	92	51.2
#3b9d	84	36	46	94	14	6	88	22	16	0	40.6

TABLE VII: Multi-task golden ticket search on two SimplEnv (WidowX) task pairs. Cells are held-out success rates (%; 300 episodes per task); Avg. is the pair average.

Task pair	Ticket	Search idx	Task 1	Task 2	Avg.
Drawer OD, CD	Base policy	—	84	65	74.5
	ticket#1	201	90.0	98.3	94.2
	ticket#2	225	97.3	98.3	97.8
Eggplant PES, PEB	Base policy	—	21	63	42.0
	ticket#1	382	60.7	88.3	74.5
	ticket#2	410	64.0	82.3	73.2

(20 envs total) and the task-averaged search-environment return as the objective; the top-ranked tickets returned by CEM are then re-evaluated on 300 held-out episodes per task (Table VII). All four tickets improve on the base policy on both tasks of their pair: the drawer tickets lift the 2-task average from 74.5 to up to 97.8, and the eggplant tickets from 42.0 to up to 74.5.